11.4 HTTP 层

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~160,000 tokens，输出 ~12,000 tokens（本章合计）

除了 WebSocket，Gateway 还提供 HTTP 端点来支持外部集成、兼容 API 和 Web 界面。HTTP 层与 WebSocket 共享同一个端口（默认 18789），通过 URL 路径区分。

11.4.1 HTTP 服务器搭建

`src/gateway/server-http.ts` 是 HTTP 路由的核心文件。它使用 Node.js 原生的 `http` / `https` 模块创建服务器：

```
// src/gateway/server-http.ts（关键导入）
import {
  createServer as createHttpServer,
  type IncomingMessage,
  type ServerResponse,
} from "node:http";
import { createServer as createHttpsServer } from "node:https";
```

HTTP 请求根据 URL 路径进行分发：

```
// HTTP 路由分发逻辑（简化示意）
function handleRequest(req: IncomingMessage, res: ServerResponse) {
  const url = new URL(req.url, `http://${req.headers.host}`);
  const pathname = url.pathname;

  // OpenAI 兼容 API
  if (pathname.startsWith("/v1/")) {
    return handleOpenAiHttpRequest(req, res, options);
  }

  // Open Responses API
  if (pathname.startsWith("/responses")) {
    return handleOpenResponsesHttpRequest(req, res, options);
  }

  // 工具调用 HTTP 接口
  if (pathname.startsWith("/tools/")) {
    return handleToolsInvokeHttpRequest(req, res, options);
  }

  // Slack HTTP 回调
  if (pathname.startsWith("/slack/")) {
    return handleSlackHttpRequest(req, res);
  }

  // Canvas / A2UI
  if (isCanvasPath(pathname)) {
    return handleA2uiHttpRequest(req, res, canvasHandler);
  }

  // Webhook 端点
  if (pathname.startsWith("/hook/")) {
    return handleWebhookRequest(req, res, hookDispatchers);
  }

  // 控制台 UI
  if (pathname === "/" || pathname.startsWith("/ui/")) {
    return handleControlUiHttpRequest(req, res, uiState);
  }

  // 头像 API
  if (pathname.startsWith("/avatar")) {
    return handleControlUiAvatarRequest(req, res, cfg);
```

```
  }
}
```

HTTP 路由采用了简单直接的路径匹配方式，没有使用 Express 或 Hono 等框架的路由系统——Gateway 的 HTTP 路由相对简单，不需要中间件链等复杂功能。

11.4.2 OpenAI 兼容 HTTP 接口

`src/gateway/openai-http.ts` 实现了与 OpenAI Chat Completions API 兼容的 HTTP 端点。这意味着任何支持 OpenAI API 的客户端（如 Cursor、Continue 等）都可以直接连接 OpenClaw Gateway 作为 LLM 后端。

端点： `POST /v1/chat/completions`

```
// src/gateway/openai-http.ts（关键类型）
type OpenAiChatCompletionRequest = {
  model?: unknown;        // 模型标识（可选，使用 Gateway 默认模型）
  stream?: unknown;       // 是否流式
  messages?: unknown;     // 消息列表
  user?: unknown;         // 用户标识
};
```

请求处理流程：

1. **认证**：使用 Bearer Token 进行认证（与 Gateway WebSocket Token 相同）

```
const token = getBearerToken(req);
const authResult = authorizeGatewayConnect({
  auth: { token },
  resolvedAuth: options.auth,
  // ...
});
```

2. **消息转换**：将 OpenAI 格式的消息列表转换为 OpenClaw 内部格式

```
function extractTextContent(content: unknown): string {
  if (typeof content === "string") return content;
  if (Array.isArray(content)) {
    return content
      .map((part) => {
        if (part.type === "text" && typeof part.text === "string")
          return part.text;
        return "";
      })
      .join("");
  }
  return "";
}
```

3. **调用 Agent**：通过内部的 `agentCommand()` 函数执行 AI 推理
4. **响应格式化**：
   - **非流式**：返回标准 JSON 响应
   - **流式**：使用 Server-Sent Events (SSE) 逐步返回

```
// SSE 流式写入
function writeSse(res: ServerResponse, data: unknown) {
  res.write(`data: ${JSON.stringify(data)}\n\n`);
}
```

SSE 格式遵循 OpenAI 的流式响应规范：每个事件以 `data:` 前缀，以 `\n\n` 分隔，最后以 `data: [DONE]\n\n` 结束。

11.4.3 Open Responses API

`src/gateway/openresponses-http.ts` 实现了 OpenAI 的 **Responses API**——这是 OpenAI 较新的 API 格式，与传统的 Chat Completions API 有所不同。

端点： `POST /responses`

这个接口主要用于支持使用 Responses API 格式的客户端和工具。实现逻辑与 Chat Completions 类似，但请求/响应格式不同。

11.4.4 工具调用 HTTP 接口

`src/gateway/tools-invoke-http.ts` 提供了通过 HTTP 直接调用 Gateway 工具的能力：

端点： `POST /tools/invoke`

这个接口允许外部系统（如 CI/CD 流水线、自动化脚本）直接调用 OpenClaw 的工具，而不需要通过 AI Agent。例如：

```
# 通过 HTTP 直接调用工具
curl -X POST http://127.0.0.1:18789/tools/invoke \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"tool": "bash", "params": {"command": "ls -la"}}'
```

其他 HTTP 端点

Webhook 端点： `/hook/:hookId`

处理来自外部服务的 Webhook 回调（如 Gmail Pub/Sub 推送）。 `src/gateway/hooks.ts` 解析 Webhook 负载，根据钩子配置将消息注入到对应的会话中。

Slack HTTP 回调： `/slack/*`

Slack 的某些功能（如交互组件、Events API）需要 HTTP 回调。 `src/slack/http/` 目录处理这些请求。

控制台 UI： `/` 和 `/ui/*`

`src/gateway/control-ui.ts` 将构建好的 Web UI 静态文件托管在根路径。用户在浏览器中访问 `http://127.0.0.1:18789/` 就能看到控制面板。

头像 API： `/avatar`

`handleControlUiAvatarRequest` 返回 AI 助手的头像图片（如果配置了的话）。

Canvas/A2UI： `/a2ui/*` 、 `/canvas/*`

Canvas 和 A2UI 的 HTTP 端点由 `src/canvas-host/a2ui.ts` 处理，提供 Agent 推送的 HTML 内容。

HTTP 安全

所有 HTTP 端点都经过认证检查。 `src/gateway/http-common.ts` 提供了通用的 HTTP 辅助函数：

```
// 发送未认证响应
export function sendUnauthorized(res: ServerResponse, message?: string) {
  res.statusCode = 401;
  res.setHeader("Content-Type", "application/json");
  res.end(JSON.stringify({ error: message ?? "Unauthorized" }));
}

// 设置 SSE 响应头
export function setSseHeaders(res: ServerResponse) {
  res.writeHead(200, {
    "Content-Type": "text/event-stream",
    "Cache-Control": "no-cache",
    "Connection": "keep-alive",
  });
}
```

`src/gateway/http-utils.ts` 提供了从请求中提取 Bearer Token、解析 Agent ID 和 Session Key 等工具函数。

到这里，我们已经完整分析了 Gateway 的服务器架构——从启动流程到 WebSocket 实现，再到 HTTP 层。Gateway 作为控制平面的角色现在应该非常清晰了：它是一个多协议（WebSocket + HTTP）的服务器，协调所有子系统的工作。

下一章我们将深入 Gateway 的协议设计和类型系统——这是理解 Gateway 如何保证通信正确性的关键。